부록 · 리눅스 명령어

Linux sed · awk · tee 자동화 실전 100제

JSP · Java · Kotlin · Spring Boot · MariaDB · Linux 서버 관리

이 문서에서 고친 곳

- 여러 파일을 awk 로 훑으며 줄 번호를 NR 로 찍던 28곳을 FNR 로 고쳤습니다. NR 은 파일을 넘어 이어지는 번호라 두 번째 파일부터 줄 번호가 틀립니다(실측: 두 번째 파일 첫 줄이 3 으로 나옴).

학습 개요

sed는 치환·삭제·삽입, awk는 조건 검색·필드 처리·계산·통계, tee는 화면 출력과 파일 저장을 동시에 수행함. 이 문서는 세 명령을 find, grep, cron 등과 결합하여 실제 운영 자동화에 적용하는 100개 예제를 정리함.

안전 원칙: 검색 → 대상 확인 → 백업 → 수정 → 재검증 → tee로 결과 저장 순서를 권장함.

1. sed 중심 파일·설정 변경 자동화

1. Spring Boot 포트 변경 + 결과 저장

```bash
FILE="application.properties"
sed -i.bak 's/^server\.port=.*/server.port=8081/' "$FILE"
awk '/^server\.port=/{print}' "$FILE" | tee port-change.log
```

2. localhost를 서버 IP로 변경

```bash
sed -i.bak 's/localhost/SERVER_IP/g' application.properties
awk '/192\.168\.0\.50/{print}' application.properties | tee change.log
```

3. MariaDB IP 변경

```bash
sed -i.bak 's/192\.168\.0\.10/SERVER_IP/g' application.properties
awk '/jdbc:mariadb/{print}' application.properties | tee mariadb-change.log
```

4. JSP 회원가입 문구 변경

```bash
sed -i.bak 's/회원가입/회원등록/g' member.jsp
awk '/회원등록/{print NR ":" $0}' member.jsp | tee member-change.log
```

5. DEBUG 줄 자동 삭제

```bash
sed -i.bak '/DEBUG/d' application.properties
awk '/DEBUG/{print NR ":" $0}' application.properties | tee debug-check.log
```

6. 빈 줄 자동 제거

```bash
sed '/^[[:space:]]*$/d' application.properties | tee application.clean.properties
```

7. 주석 줄 제거 결과 저장

```bash
sed '/^[[:space:]]*#/d' application.properties | tee no-comment.properties
```

8. 10번째 줄 삭제

```bash
cp member.jsp member.jsp.bak
sed -i '10d' member.jsp
awk 'NR>=8&&NR<=12{print NR ":" $0}' member.jsp | tee line-check.txt
```

9. 10~20번째 줄 삭제

```bash
cp member.jsp member.jsp.bak
sed -i '10,20d' member.jsp
awk 'NR>=8&&NR<=15{print NR ":" $0}' member.jsp | tee range-check.txt
```

10. server.port 뒤에 server.address 삽입

```bash
sed -i.bak '/^server\.port=/a server.address=0.0.0.0' application.properties
awk '/server\.port|server\.address/{print}' application.properties | tee server-config.txt
```

11. datasource 앞에 설명 주석 삽입

```bash
sed -i.bak '/^spring\.datasource/i # Database Configuration' application.properties
sed -n '/Database Configuration/,+3p' application.properties | tee db-section.txt
```

12. JSP title 변경

```bash
sed -i.bak 's#<title>.*</title>#<title>WizData</title>#' index.jsp
awk '/<title>/{print NR ":" $0}' index.jsp | tee title.log
```

13. JSP DEBUG 블록 삭제

```bash
sed -i.bak '/<!-- DEBUG/,/DEBUG -->/d' index.jsp
awk '/DEBUG/{print NR ":" $0}' index.jsp | tee debug-remain.log
```

14. 여러 공백 정리

```bash
sed 's/[[:space:]][[:space:]]*/ /g' file.txt | tee clean.txt
```

15. 앞 공백 제거

```bash
sed 's/^[[:space:]]*//' file.txt | tee left-trim.txt
```

16. 뒤 공백 제거

```bash
sed 's/[[:space:]]*$//' file.txt | tee right-trim.txt
```

17. 앞뒤 공백 동시 제거

```bash
sed 's/^[[:space:]]*//;s/[[:space:]]*$//' file.txt | tee trim.txt
```

18. datasource 설정만 추출

```bash
sed -n '/spring\.datasource/p' application.properties | tee db-config.txt
```

19. JSP 100~150줄 추출

```bash
sed -n '100,150p' member.jsp | awk '{print NR+99 ":" $0}' | tee jsp-code.txt
```

20. properties 파일 포트 일괄 변경

```bash
find . -name "*.properties" -exec sed -i.bak 's/^server\.port=.*/server.port=8081/' {} +
find . -name "*.properties" -exec awk '/server\.port/{print FILENAME ":" FNR ":" $0}' {} + | tee ports.txt
```

2. awk 중심 검색·분석·통계 자동화

21. ERROR 자동 검색

```bash
awk '/ERROR/{print NR ":" $0}' app.log | tee error.log
```

22. WARN 자동 검색

```bash
awk '/WARN/{print NR ":" $0}' app.log | tee warn.log
```

23. ERROR + WARN 동시 검색

```bash
awk '/ERROR|WARN/{print NR ":" $0}' app.log | tee warning.log
```

24. ERROR 개수 자동 계산

```bash
awk '/ERROR/{n++} END{print "ERROR =",n+0}' app.log | tee error-count.txt
```

25. ERROR/WARN 통계

```bash
awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR =",e+0; print "WARN =",w+0}' app.log | tee statistics.txt
```

26. JSP form 부분 추출

```bash
awk '/<form/,/<\/form>/{print NR ":" $0}' member.jsp | tee form.txt
```

27. JSP script 부분 추출

```bash
awk '/<script/,/<\/script>/{print NR ":" $0}' member.jsp | tee script.txt
```

28. JSP 스크립틀릿 부분 추출

```bash
awk '/<%/,/%>/{print NR ":" $0}' member.jsp | tee jsp-java.txt
```

29. 50~100줄 추출

```bash
awk 'NR>=50&&NR<=100{print NR ":" $0}' file.txt | tee range.txt
```

30. Linux 사용자명 추출

```bash
awk -F: '{print $1}' /etc/passwd | tee users.txt
```

31. 사용자명 + UID 추출

```bash
awk -F: '{print $1,$3}' /etc/passwd | tee user-uid.txt
```

32. UID 1000 이상 계정 찾기

```bash
awk -F: '$3>=1000{print $1,$3}' /etc/passwd | tee normal-users.txt
```

33. CSV 첫 번째 열 추출

```bash
awk -F, '{print $1}' data.csv | tee col1.txt
```

34. CSV 첫 번째·세 번째 열 추출

```bash
awk -F, '{print $1,$3}' data.csv | tee selected-columns.txt
```

35. 숫자 합계 계산

```bash
awk '{sum+=$1} END{print "TOTAL =",sum}' data.txt | tee total.txt
```

36. 숫자 평균 계산

```bash
awk '{sum+=$1} END{if(NR) print "AVERAGE =",sum/NR}' data.txt | tee average.txt
```

37. 세 번째 필드 80 이상 추출

```bash
awk '$3>=80{print}' data.txt | tee high.txt
```

38. 여러 로그에서 ERROR 위치 출력

```bash
awk '/ERROR/{print FILENAME ":" FNR ":" $0}' *.log | tee all-error.txt
```

39. Java TODO 자동 검색

```bash
find src -name "*.java" -exec awk '/TODO/{print FILENAME ":" FNR ":" $0}' {} + | tee java-todo.txt
```

40. Kotlin TODO 자동 검색

```bash
find src -name "*.kt" -exec awk '/TODO/{print FILENAME ":" FNR ":" $0}' {} + | tee kotlin-todo.txt
```

3. tee 중심 로그·보고서 자동화

41. 디스크 상태 보고서

```bash
df -h | tee disk.txt
```

42. 날짜를 기존 로그에 추가

```bash
date | tee -a server.log
```

43. 메모리 상태 보고서

```bash
free -h | tee memory.txt
```

44. 프로세스 전체 저장

```bash
ps aux | tee process.txt
```

45. Java 프로세스 저장

```bash
pgrep -af java | tee java-process.txt
```

46. 열린 포트 상태 저장

```bash
ss -tulpn | tee ports.txt
```

47. 8080 포트만 저장

```bash
ss -tulpn | awk '/:8080/{print}' | tee port8080.txt
```

48. 서버 종합 기본 보고서

```bash
{ date; uptime; free -h; df -h; } | tee server-report.txt
```

49. 한 결과를 두 파일에 동시 저장

```bash
echo "SERVER OK" | tee server1.log server2.log
```

50. application.properties 끝에 설정 추가

```bash
echo "server.port=8080" | tee -a application.properties
```

51. root 설정파일 작성

```bash
echo "TEST=1" | sudo tee /etc/test.conf
```

52. Spring Boot 실행 로그 저장

```bash
java -jar app.jar 2>&1 | tee app.log
```

53. Spring Boot 실행 로그 누적

```bash
java -jar app.jar 2>&1 | tee -a app.log
```

54. HTTP 응답 저장

```bash
curl -s http://localhost:8080 | tee response.html
```

55. 환경변수 목록 저장

```bash
env | tee env.txt
```

4. JSP · Java · Kotlin 코드 분석 자동화

56. JSP request.getParameter 검색

```bash
find . -name "*.jsp" -exec awk '/request\.getParameter/{print FILENAME ":" FNR ":" $0}' {} + | tee request-check.txt
```

57. JSP session 사용 검색

```bash
find . -name "*.jsp" -exec awk '/session\./{print FILENAME ":" FNR ":" $0}' {} + | tee session-check.txt
```

58. JSP System.out.println 검색

```bash
find . -name "*.jsp" -exec awk '/System\.out\.println/{print FILENAME ":" FNR ":" $0}' {} + | tee jsp-println.txt
```

59. JSP TODO 검색

```bash
find . -name "*.jsp" -exec awk '/TODO/{print FILENAME ":" FNR ":" $0}' {} + | tee jsp-todo.txt
```

60. Java System.out.println 검색

```bash
find src -name "*.java" -exec awk '/System\.out\.println/{print FILENAME ":" FNR ":" $0}' {} + | tee java-print.txt
```

61. Kotlin println 검색

```bash
find src -name "*.kt" -exec awk '/println\(/{print FILENAME ":" FNR ":" $0}' {} + | tee kotlin-print.txt
```

62. Java public 메서드 검색

```bash
find src -name "*.java" -exec awk '/public .*[[:space:]]*[A-Za-z0-9_]+\(/{print FILENAME ":" FNR ":" $0}' {} + | tee java-methods.txt
```

63. Kotlin fun 검색

```bash
find src -name "*.kt" -exec awk '/^[[:space:]]*fun /{print FILENAME ":" FNR ":" $0}' {} + | tee kotlin-functions.txt
```

64. Repository 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/Repository/{print FILENAME ":" FNR ":" $0}' {} + | tee repositories.txt
```

65. @Entity 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/@Entity/{print FILENAME ":" FNR ":" $0}' {} + | tee entities.txt
```

66. DTO 관련 코드 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/Dto|DTO/{print FILENAME ":" FNR ":" $0}' {} + | tee dtos.txt
```

67. Controller 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/@RestController|@Controller/{print FILENAME ":" FNR ":" $0}' {} + | tee controllers.txt
```

68. Service 검색

```bash
find src -type f \( -name "*.java" -o -name "*.kt" \) -exec awk '/@Service/{print FILENAME ":" FNR ":" $0}' {} + | tee services.txt
```

69. JSP href 검색

```bash
find . -name "*.jsp" -exec awk '/href=/{print FILENAME ":" FNR ":" $0}' {} + | tee href.txt
```

70. JSP form action 검색

```bash
find . -name "*.jsp" -exec awk '/action=/{print FILENAME ":" FNR ":" $0}' {} + | tee actions.txt
```

5. Spring Boot 설정·로그 자동화

71. server.port 검색

```bash
awk '/server\.port/{print NR ":" $0}' application.properties | tee port.txt
```

72. datasource.url 검색

```bash
awk '/datasource\.url/{print NR ":" $0}' application.properties | tee database.txt
```

73. JPA 설정 추출

```bash
awk '/spring\.jpa/{print NR ":" $0}' application.properties | tee jpa.txt
```

74. Spring profile 추출

```bash
awk '/spring\.profiles/{print NR ":" $0}' application.properties | tee profile.txt
```

75. 포트 변경 후 자동 검증

```bash
sed -i.bak 's/^server\.port=.*/server.port=8081/' application.properties
awk '/server\.port/{print}' application.properties | tee port-result.txt
```

76. DB IP 변경 후 자동 검증

```bash
sed -i.bak 's/192\.168\.0\.10/SERVER_IP/g' application.properties
awk '/datasource\.url/{print}' application.properties | tee database-result.txt
```

77. ddl-auto를 update로 변경

```bash
sed -i.bak 's/^spring\.jpa\.hibernate\.ddl-auto=.*/spring.jpa.hibernate.ddl-auto=update/'
application.properties
awk '/ddl-auto/{print}' application.properties | tee ddl.txt
```

78. show-sql을 true로 변경

```bash
sed -i.bak 's/^spring\.jpa\.show-sql=.*/spring.jpa.show-sql=true/' application.properties
awk '/show-sql/{print}' application.properties | tee show-sql.txt
```

79. root logging level을 INFO로 변경

```bash
sed -i.bak 's/^logging\.level\.root=.*/logging.level.root=INFO/' application.properties
awk '/logging\.level\.root/{print}' application.properties | tee logging.txt
```

80. application.yml 포트 변경

```bash
sed -i.bak '/^[[:space:]]*port:/s/[0-9][0-9]*/8081/' application.yml
awk '/^[[:space:]]*port:/{print NR ":" $0}' application.yml | tee yml-port.txt
```

81. Spring Boot ERROR 통계

```bash
awk '/ERROR/{e++} END{print "ERROR:",e+0}' app.log | tee spring-error.txt
```

82. 최근 ERROR 20건 저장

```bash
awk '/ERROR/{print NR ":" $0}' app.log | tail -20 | tee last-error.txt
```

83. 최근 WARN 20건 저장

```bash
awk '/WARN/{print NR ":" $0}' app.log | tail -20 | tee last-warn.txt
```

84. Exception 자동 검색

```bash
awk '/Exception/{print NR ":" $0}' app.log | tee exceptions.txt
```

85. Spring Boot 로그 종합 통계

```bash
awk '/ERROR/{e++} /WARN/{w++} /Exception/{x++} END{print "ERROR =",e+0; print "WARN =",w+
0; print "Exception =",x+0}' app.log | tee boot-stat.txt
```

6. Linux 서버 상태 점검 자동화

86. 디스크 80% 이상 경고

```bash
df -P | awk 'NR>1{u=$5; gsub("%","",u); if(u>=80) print "WARNING:",$1,u"%",$6}' | tee disk
-warning.txt
```

87. 메모리 사용량 보고서

```bash
free -m | awk 'NR==2{print "Total:",$2,"MB"; print "Used:",$3,"MB"; print "Free:",$4,"MB"}' | tee memory-report.txt
```

88. CPU load 기록

```bash
uptime | awk '{print}' | tee load.txt
```

89. Java 프로세스 자동 점검

```bash
pgrep -af java | awk '{print "JAVA:",$0}' | tee java-status.txt
```

90. 8080 포트 점검

```bash
ss -tulpn | awk '/:8080/{print}' | tee 8080-status.txt
```

91. 3306 MariaDB 포트 점검

```bash
ss -tulpn | awk '/:3306/{print}' | tee mariadb-port.txt
```

92. SSH 로그인 기록 분석

```bash
last | awk '{print $1,$3,$4,$5,$6}' | tee login-report.txt
```

93. root 로그인 기록만 추출

```bash
last | awk '$1=="root"{print}' | tee root-login.txt
```

94. 홈 디렉터리 용량 보고서

```bash
du -sh /home/* 2>/dev/null | sort -h | tee home-size.txt
```

95. 서버 기본 상태 통합 저장

```bash
{ date; hostname; uptime; df -h; free -h; } | tee server-status.txt
```

7. 종합 Shell Script 자동화

96. Spring Boot 자동 상태 보고서

```bash
#!/bin/bash
LOG="app.log"
{
 echo "===== SERVER REPORT ====="
 date
 echo "[JAVA]"
 pgrep -af java
 echo "[PORT]"
 ss -tulpn | awk '/:8080/'
 echo "[ERROR]"
 awk '/ERROR/{print NR ":" $0}' "$LOG" | tail -10
 echo "[ERROR COUNT]"
 awk '/ERROR/{e++} END{print e+0}' "$LOG"
} | tee server-report.txt
```

97. 설정파일 자동 백업 + 변경

```bash
#!/bin/bash
FILE="application.properties"
cp "$FILE" "$FILE.$(date +%Y%m%d-%H%M%S).bak"
sed -i 's/^server\.port=.*/server.port=8081/' "$FILE"
awk '/server\.port/{print "NEW CONFIG:",$0}' "$FILE" | tee configuration-change.txt
```

98. JSP 자동 코드 품질 점검

```bash
#!/bin/bash
find . -name "*.jsp" -exec awk '
/System\.out\.println/{print "[PRINT]",FILENAME,FNR,$0}
/DEBUG/{print "[DEBUG]",FILENAME,FNR,$0}
/TODO/{print "[TODO]",FILENAME,FNR,$0}
/request\.getParameter/{print "[PARAMETER]",FILENAME,FNR,$0}
' {} + | tee jsp-check.txt
```

99. Spring Boot 프로젝트 자동 검사

```bash
#!/bin/bash
{
 echo "===== PORT ====="
 find src/main/resources -type f \( -name "*.properties" -o -name "*.yml" \)    -exec awk '/se
rver\.port|port:/{print FILENAME ":" FNR ":" $0}' {} +
 echo "===== DATABASE ====="
 find src/main/resources -type f    -exec awk '/datasource|mariadb|jdbc/{print FILENAME ":" FN
R ":" $0}' {} +
 echo "===== CONTROLLER ====="
 find src -type f \( -name "*.java" -o -name "*.kt" \)    -exec awk '/@RestController|@Control
ler/{print FILENAME ":" FNR ":" $0}' {} +
 echo "===== TODO ====="
 find src -type f \( -name "*.java" -o -name "*.kt" -o -name "*.jsp" \)    -exec awk '/TODO/{p
rint FILENAME ":" FNR ":" $0}' {} +
} | tee project-check.txt
```

100. sed + awk + tee 서버 종합 자동화

```bash
#!/bin/bash
PROJECT="/home/devuser/app"
LOG="$PROJECT/app.log"
CONFIG="$PROJECT/application.properties"
REPORT="$PROJECT/daily-report-$(date +%Y%m%d).txt"
cd "$PROJECT" || exit 1
{
 echo "========================================"
 echo "      SERVER AUTOMATION REPORT"
 echo "========================================"
 date
 echo "===== HOST ====="
 hostname
 echo "===== JAVA PROCESS ====="
 pgrep -af java || echo "Java process 없음"
 echo "===== PORT 8080 ====="
 ss -tulpn | awk '/:8080/{print}'
 echo "===== PORT 3306 ====="
 ss -tulpn | awk '/:3306/{print}'
 echo "===== SPRING CONFIG ====="
 awk '/server\.port/{print "[PORT]",$0} /datasource\.url/{print "[DATABASE]",$0} /spring\.jpa/
{print "[JPA]",$0}' "$CONFIG"
 echo "===== LOG STATISTICS ====="
 awk '/ERROR/{e++} /WARN/{w++} /Exception/{x++} END{print "ERROR =",e+0; print "WARN =",w+0; p
rint "EXCEPTION =",x+0}' "$LOG"
 echo "===== RECENT ERRORS ====="
 awk '/ERROR/{print NR ":" $0}' "$LOG" | tail -20
 echo "===== DISK ====="
 df -P | awk 'NR>1{u=$5; gsub("%","",u); print $1,u"%",$6; if(u>=80) print ">>> WARNING: DISK
usage high"}'
 echo "===== MEMORY ====="
 free -m | awk 'NR==2{print "Total =",$2,"MB"; print "Used =",$3,"MB"; print "Free =",$4,"M
B"}'
 echo "===== TODO CODE ====="
 find src -type f \( -name "*.java" -o -name "*.kt" -o -name "*.jsp" \)    -exec awk '/TODO/{p
rint FILENAME ":" FNR ":" $0}' {} +
 echo "========================================"
 echo "             COMPLETE"
 echo "========================================"
} | tee "$REPORT"
```

8. cron 자동 실행 예제

스크립트 실행 권한을 부여한 뒤 crontab에 등록하면 정기 자동화를 구성할 수 있음.

```bash
chmod +x server-report.sh
crontab -e

# 매일 오전 9시 실행
0 9 * * * /home/devuser/app/server-report.sh

# 매일 오후 11시 로그 분석
0 23 * * * /home/devuser/app/log-report.sh
```

9. 실무 안전 체크리스트

- sed -i 실행 전에는 가능하면 -i를 빼고 화면에서 예상 결과를 먼저 확인함.
- 운영 설정파일은 sed -i.bak 또는 날짜가 포함된 별도 백업 파일을 만든 뒤 수정함.
- find + sed 일괄 변경은 대상 파일 목록을 먼저 출력하여 범위를 확인함.
- 비밀번호, API 키, 토큰 등 민감한 설정값을 tee 보고서에 그대로 기록하지 않도록 주의함.
- 자동 삭제와 자동 변경은 테스트 환경에서 검증한 후 운영 서버에 적용함.
- cron 실행 시 상대경로보다 절대경로를 사용하는 것이 안전함.
